과제 #5

M1522.006700 확장형 고성능 컴퓨팅 (001)

2021-15531 컴퓨터공학부 김인호
November 17, 2025

1 Matrix Multiplication Single GPU

- **병렬화 방식**
  $(M \times N)$ 크기의 행렬 C를 계산하기 위해, CUD의 그리드를 2차원으로 설정한다. 전체 그리드는 타일 크기 `TILE_WIDTH`에 대해 `TILE_WIDTH` × `TILE_WIDTH` 크기의 2차원 스레드 블록들로 구성된다. 각 스레드 블록은 행렬 C의 `TILE_WIDTH` × `TILE_WIDTH` 크기 타일 하나를 계산하는 역할을 한다. 블록 내의 각 스레드(`threadIdx.x`, `threadIdx.y`)는 해당 타일 내의 원소 한 개(C_{ij})를 계산한다. 각 스레드는 C_{ij}의 값을 계산하기 위해 A 행렬의 한 행과 B 행렬의 한 열에 대해 행렬곱을 수행해야 한다. 이 과정은 `TILE_WIDTH` 크기의 타일 단위로 나뉘어진 A 행렬과 B행렬의 모든 행과 열에 접근하기 위한 반복문을 돌며 수행된다.

- **성능 최적화를 위해 적용한 방법 및 고려 사항**
  - **타일링:** 행렬 A, B, C를 `TILE_WIDTH` 크기의 작은 타일로 분할하여 계산한다. 이는 데이터의 locality을 높여 캐시 및 공유 메모리의 효율성을 증가시킨다.
  - **공유 메모리(shared memory) 활용:** 각 스레드 블록은 자신이 계산할 C 타일에 필요한 A 타일과 B 타일을 전역(global) 메모리에서 공유 메모리로 옮긴다. 공유 메모리는 전역 메모리보다 훨씬 빠르므로 메모리 접근 속도에서 이익을 본다.
  - **데이터 재사용:** 공유 메모리에 옮겨진 A 타일의 각 행과 열은 해당 스레드 블록에서 행이 같은 스레드끼리, 열이 같은 스레드끼리 여러 번 사용되며 타일링의 효과를 본다.
  - **레지스터 활용:** 각 스레드는 C_{ij}의 값을 `C_val`에 누적하는데, `C_val`은 로컬 레지스터에 있으므로 접근 속도가 가장 빠르다. K번의 반복이 끝난 후에는 레지스터의 값을 전역 메모리로 한 번만 쓴다.
  - **이중 버퍼링:** 공유 메모리에 둘 A, B 타일을 `A_tile[2][][]`, `B_tile[2][][]`로 각각 2개씩 할당한다. 커널에서 (t)번째 타일을 가지고 연산을 수행하는 동시에 $(t+1)$번째 타일을 전역 메모리에서 다른 공유 메모리 세트로 미리 옮겨 놓는다. 이는 연산 시간을 메모리 접근 시간 뒤에 숨기는(overlapping) 효과를 주며 시간을 단축시킨다.

  위와 같은 최적화를 적용하여 성능을 측정한 결과, $M = N = K = 4096$일 때 1577.489761 GFLOPS (5회 측정 후 평균)의 결과를 보였다.

- `matmul_single.cu`**의 각 부분 설명**
  `matmul_initialize` 함수에서는 CUDA 디바이스(GPU)를 확인하고 GPU의 전역 메모리에 행렬 A, B, C를 저장할 공간을 할당한다. 아래의 CUDA API가 쓰인다.
  - `cudaGetDeviceCount`: 사용할 수 있는 CUDA 디바이스의 개수를 알려준다.
  - `cudaMalloc`: 디바이스(GPU)에 지정한 크기만큼의 메모리를 할당한다.

`matmul` 함수에서는 A, B 행렬의 데이터를 디바이스로 복사하고 그리드와 블록의 차원을 설정하여 `matmul_kernel`을 실행시킨 뒤 C 행렬에 결과를 담는다. 아래의 CUDA API가 쓰인다.

- `cudaMemcpy` 호스트와 디바이스 간에 (인자를 통해 방향 설정) 데이터를 복사한다.
- `cudaDeviceSynchronize`: 디바이스가 실행을 완료할 때까지 기다린다.(동기화)

`matmul_kernel` 함수는 디바이스에서 실행되는, 행렬곱에 핵심적인 역할을 하는 함수이다. A 행렬과 B 행렬을 타일 단위로 공유 메모리에 로드하면서, 각 스레드가 자신이 담당한 C 행렬의 원소에 대한 계산을 나누어 수행한다. 디바이스의 공유 메모리에 행렬의 일부분을 로드하는 과정에서, `__syncthreads` 함수가 사용되어 스레드 간의 동기화가 이루어진다.

`matmul_finalize` 함수는 `matmul_initialize` 함수에서 행렬 A, B, C를 위해 할당했던 디바이스의 모든 메모리를 해제한다.

- `cudaFree`: 디바이스에 할당된 메모리를 해제(free)한다.

- **OpenCL과 CUDA의 장점 비교**
  CUDA는 OpenCL에 비해 NVIDIA GPU의 아키텍쳐를 깊이 활용하여 최적화 수준이 높고, 생태계가 풍부하다.
  OpenCL은 CUDA에 비해 AMD, Intel 등 여러 제조사의 다양한 하드웨어에서 동작 가능해 범용성이 좋다.

- **최적화 방식 정리 및 성능 실험** 아래의 성능 실험에서는 항상 $M = N = K = 4096$으로 두고, 5회 반복하여 측정한 시간의 평균으로 성능을 구하였다.
  - **공유 메모리 활용**: 현재 코드에서는 공유 메모리를 활용하여 행렬 데이터에 접근하는 속도를 빠르게 한다. 공유 메모리를 선언하지 않고 `C_val`을 계산할 때 단순히 A, B 행렬에 직접 접근하여 계산하는 것으로 코드를 수정하여, 공유 메모리를 활용했을 때와 그렇지 않았을 때 성능을 실험하였다. 공유 메모리를 선언하지 않도록 수정한 코드에서 성능을 측정한 결과 1246.198614 GFLOPS로 나타났다. 이를 기존 코드의 성능인 1577.489761 GFLOPS와 비교하면 공유 메모리 활용은 성능을 약 27% 향상시킨다고 할 수 있다.
  - **타일링**: 현재 코드에서는 행렬 A와 B를 타일 단위로 나누어 공유 메모리에 옮기면서 행렬곱 연산을 수행한다. 현재 코드에서 설정한 타일 크기 `TILE_WIDTH`는 32로, 블록 하나에서 사용 가능한 최대 스레드 수에 맞게 설정하였다. 만약 이 타일 크기를 1로 설정한다면, 타일링을 전혀 사용하지 않는 것이나 다름없는 효과를 낼 것이다. 타일 크기를 다양하게 설정하여 성능을 실험한 결과는 아래와 같다.

`TILE_WIDTH`	1	2	4	8	16	24	32
성능 (GFLOPS)	측정 불가	48.0	299.7	968.8	1446.0	1310.8	1577.5

    타일 크기가 1일 때는 계산 노드의 시간 제한 초과로 인해 정확한 측정이 불가하였다. 타일 크기가 2의 배수인 경우, 타일 크기가 클 수록 성능이 증가하는 모습을 보였다.
  - **더블 버퍼링**: 연산과 메모리 전송을 중첩시켜, 둘 중 더 오래 걸리는 연산으로 인한 성능 저하를 줄이는 효과를 낸다. 현재 코드에서 메모리 로드와 연산 사이에 `__syncthread` 함수를 넣어 이 효과를 없앤 후 성능을 측정해보면 1553.433810 GFLOPS로 나타난다. 따라서 더블 버퍼링은 성능을 약 1.5% 향상시킨다고 할 수 있다.

2 Matrix Multiplication Multi-GPU

- **병렬화 방식**
  여러 개의 GPU로 병렬화를 하기 위해, $(M \times N)$ 크기의 행렬 C와 $(M \times K)$ 크기의 행렬 A를 행(M) 기준으로 사용 가능한 GPU의 수(`num_devices`)만큼 분할한다. 각 GPU가 할당받는 행렬 C와 A의 행

구간을 M_{local}이라 하자. 각 GPU는 C의 $(M_{\text{local}} \times N)$ 부분을 계산하기 위해 A의 $(M_{\text{local}} \times K)$ 부분과 행렬 B 전체$(K \times N)$를 할당받는다. 호스트(CPU)는 `#pragma omp parallel for`를 사용하여 OpenMP 스레드를 생성하며, 스레드 하나가 GPU 하나를 제어한다. 각 GPU 내에서의 커널(`matmul_kernel`) 병렬화 방식은 Single GPU의 코드와 동일하다. 즉, 각 GPU는 할당받은 $(M_{\text{local}} \times N)$ 크기의 C 부분 행렬을 계산하기 위해 이차원 그리드를 생성하며, 각 스레드 블록은 `TILE_WIDTH` $\times$ `TILE_WIDTH` 크기의 C 타일을 계산한다.

- **성능 최적화를 위해 적용한 방법 및 고려 사항**
  - **데이터 병렬화(DP):** 행을 기준으로 해야 할 계산 작업을 분할하여 여러 GPU에 나눈다. 각 GPU는 병렬적으로 행렬 C의 나뉘어진 부분을 계산한다.
  - **비동기 API 호출:** 호스트 코드가 여러 개의 GPU를 대상으로 하는 명령들 동시에 실행하기 위해 비동기 호출을 사용한다. 각 GPU가 실행할 명령을 담을 스트림을 생성하고, `matmul` 함수의 각 API를 스트림에 넣어 비동기적으로 API를 실행시킨다.
  - **타일링, 공유 메모리 활용, 이중 버퍼링:** 각 GPU에서 실행되는 커널은 Single GPU 버전과 동일하게 Tiling, Shared Memory, Double Buffering 최적화가 모두 적용되어 있어, GPU 하나하나의 연산 효율을 높인다.

  위와 같은 최적화를 적용하여 성능을 측정한 결과, $M = 16384, N = K = 4096$일 때 8745.296001 GFLOPS(5회 측정 후 평균)라는 결과를 보였다.

- `matmul_multi.cu`**의 각 부분 설명**
  `matmul_initialize` 함수는 CUDA 디바이스를 개수를 세고 각 GPU의 이름을 확인한 뒤, 각 GPU에 할당할 행렬 A, C의 행 구간을 설정하고 행렬 데이터를 저장할 공간을 각 GPU의 전역 메모리에 할당한다. 아래의 CUDA API가 쓰인다.
  - `cudaGetDeviceCount`: 사용할 수 있는 CUDA 디바이스의 개수를 알려준다.
  - `cudaGetDeviceProperties`: 디바이스의 정보들(이름, 사양 등)을 가져온다.
  - `cudaSetDevice`: 호스트 스레드에서 명령에 사용할 디바이스를 설정한다.
  - `cudaMalloc`: 디바이스(GPU)에 지정한 크기만큼의 메모리를 할당한다.
  - `cudaStreamCreate`: 비동기 처리를 위한 스트림을 만든다.

  `matmul` 함수는 각각의 GPU에서 계산할 A 행렬의 일부분과 B 행렬 전체를 각 GPU로 복사하고, 그리드와 블록의 차원을 설정하여 `matmul_kernel`을 실행시킨 뒤, 해당 GPU가 담당한 C 행렬의 일부분에 결과를 담는다. 여기에 쓰이는 API 중 앞에서 설명하지 않은 것들을 아래에서 설명한다.
  - `cudaMemcpyAsync`: 호스트와 디바이스 간에 (인자를 통해 방향 설정) 데이터를 복사하는 명령을 스트림에 넣고 비동기적으로 실행시킨다.
  - `cudaDeviceSynchronize`: 디바이스가 실행을 완료할 때까지 기다린다.

  `matmul_kernel` 함수는 Single GPU에서 사용한 커널 함수와 같다. Multi-GPU에서 다른 점은 계산하는 행렬의 행을 각 GPU마다 나누어 담당한다는 것인데, 이는 함수의 인자로 주는 A, C, M을 조절하여 구현할 수 있다.

  `matmul_finalize` 함수는 각각의 GPU에 대해 행렬 데이터를 담기 위해 할당했던 모든 메모리와 스트림 자원을 해제한다.
  - `cudaFree`: 디바이스에 할당된 메모리를 해제(free)한다.
  - `cudaStreamDestroy`: 생성했던 비동기 스트림을 제거한다.

- **최적화 방식 정리 및 성능 실험** 아래의 성능 실험에서는 특별한 언급이 없다면 $M = 16384, N = K = 4096$으로 두고 5회 반복하여 측정한 시간의 평균으로 성능을 구하였다.

– **Multi-GPU를 활용한 데이터 병렬화:** 작업을 M차원(결과를 담을 행렬의 행) 기준으로 분할하여 `num_devices`개의 GPU에서 동시에 처리한다. 이론적으로 GPU 수에 비례하는 성능 향상을 기대할 수 있다. 직접 GPU의 개수를 조절하며 성능을 측정한 결과는 아래와 같았다.

GPU 개수	1	2	3	4
성능(GFLOPS)	2133.3	4330.2	6535.4	8745.3

실험 결과, 실제로 성능이 GPU의 개수와 거의 비례하는 것을 확인할 수 있었다.

– **타일링과 공유 메모리 활용:** Single-GPU에서와 마찬가지로, 커널 내 공유 메모리 활용과 타일링은 메모리 접근 속도를 향상시키는 가장 핵심적인 최적화이다. 위에서와 마찬가지로 공유 메모리를 사용하지 않고 성능을 실험하였다. 그 결과 6546.735852 GFLOPS이 나와, 공유 메모리 활용은 약 34%의 성능 향상 효과가 있다고 할 수 있다.

커널에서 공유 메모리에 저장하는 A 타일과 B 타일의 한 행과 열의 길이인 `TILE_SIZE`의 크기도 조절하며 실험하였다.

`TILE_WIDTH`	1	2	4	8	16	24	32
성능 (GFLOPS)	31.9	244.0	1499.7	5157.3	7890.1	7144.6	8745.3

타일 크기가 1일 때는 시간상의 한계로 첫 번째 반복의 데이터만을 바탕으로 성능을 계산하였다. 타일 크기가 2의 배수인 경우, 타일 크기가 클수록 성능이 증가하는 모습을 보였다.

– **비동기 API 호출:** `cudaMemcpyAsync`와 스트림을 사용하여 각각의 GPU로 행렬 데이터를 복사하고 커널을 실행하는 명령을 동시에 내릴 수 있도록 하였다. 비동기 API를 사용하지 않고, 동시에 GPU에게 명령을 내리는 다른 방법으로는 OpenMP가 있다. 비교를 위해 OpenMP와 비동기 API의 유무를 기존으로 4가지 경우로 나누어 성능을 실험하였다. OpenMP는 각 `matmul` 함수에서 각 디바이스에 대한 반복문 위에 `#pragma omp parallel for`을 넣어 사용하였다. 현재의 코드는 OpenMP는 사용하지 않고 비동기 API만 사용하는 경우이다.

	OpenMP 미사용	OpenMP 사용
비동기 API 미사용	1747.2	6317.6
비동기 API 사용	8745.3	6317.0

(성능은 GFLOPS 단위로 표기)

OpenMP를 사용한 상황에서의 비동기 API 사용은 효과가 없었고, OpenMP보다는 비동기 API가 더 큰 성능 향상을 보였다.